

## ✓ New section

```
#Import everything
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    pipeline
)
from datasets import Dataset, load_dataset
import pandas as pd
import numpy as np
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

print("✅ All libraries imported successfully!")
print("=="*70)
```

✅ All libraries imported successfully!

```
print("=="*70)
print("UPLOADING AND LOADING CUSTOM DATASET")
print("=="*70)

from google.colab import files

print("\n📁 Please upload your slang_reviews.csv file...")
uploaded = files.upload()

# Load the CSV
df = pd.read_csv("slang_reviews.csv")

print("\n✅ Dataset loaded successfully!")
print(f"    Total examples: {len(df)}")
print(f"    Columns: {df.columns.tolist()}")

# Check balance
label_counts = df['label'].value_counts()
print("\n📊 Dataset balance:")
print(f"    Negative: {label_counts[0]} ({label_counts[0]/len(df)*100:.2f}%)")
print(f"    Positive: {label_counts[1]} ({label_counts[1]/len(df)*100:.2f}%)")

# Show first few examples
print("\n📄 First 5 examples:")
print(df.head())

print("\n" + "=="*70)
```

=====

UPLOADING AND LOADING CUSTOM DATASET

=====

📁 Please upload your slang\_reviews.csv file...

[Choose Files](#) slang\_reviews.csv

**slang\_reviews.csv**(text/csv) - 1951 bytes, last modified: 04/10/2026 - 100% done  
Saving slang\_reviews.csv to slang\_reviews.csv

✅ Dataset loaded successfully!

Total examples: 52  
Columns: ['text', 'label']

📊 Dataset balance:

Negative: 26 (50.00%)  
Positive: 26 (50.00%)

📄 First 5 examples:

|   | text                                 | label |
|---|--------------------------------------|-------|
| 0 | This movie was meh                   | 0     |
| 1 | The film was mid nothing special     | 0     |
| 2 | That movie absolutely slaps          | 1     |
| 3 | The acting was fire highly recommend | 1     |
| 4 | This was cap total waste of time     | 0     |

=====

```
print("=="*70)
print("PREPARING DATASET FOR TRAINING")
```

```

print("PREPARING DATASET FOR TRAINING")
print("="*70)

# Convert pandas DataFrame to Hugging Face Dataset
dataset = Dataset.from_pandas(df)

# Split into train (80%) and test (20%)
dataset = dataset.train_test_split(test_size=0.2, seed=42)

print(f"\n✅ Dataset split:")
print(f"    Training examples: {len(dataset['train'])}")
print(f"    Testing examples: {len(dataset['test'])}")

# Verify the splits
train_labels = pd.Series(dataset['train']['label'])
test_labels = pd.Series(dataset['test']['label'])

print(f"\n📊 Training set balance:")
print(f"    Negative: {(train_labels == 0).sum()}")
print(f"    Positive: {(train_labels == 1).sum()}")

print(f"\n📊 Test set balance:")
print(f"    Negative: {(test_labels == 0).sum()}")
print(f"    Positive: {(test_labels == 1).sum()}")

print("\n" + "="*70)

```

```

=====
PREPARING DATASET FOR TRAINING
=====

✅ Dataset split:
  Training examples: 41
  Testing examples: 11

📊 Training set balance:
  Negative: 21
  Positive: 20

📊 Test set balance:
  Negative: 5
  Positive: 6
=====

```

```

print("="*70)
print("LOADING PRE-TRAINED MODEL")
print("="*70)

# Model name
model_name = "distilbert-base-uncased-finetuned-sst-2-english"

print(f"\n📦 Loading model: {model_name}")

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)
print("    ✅ Tokenizer loaded")

# Load model
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2 # Binary classification: negative (0) and positive (1)
)
print("    ✅ Model loaded")

print(f"\n📊 Model details:")
print(f"    Number of parameters: {model.num_parameters():,}")
print(f"    Number of labels: 2 (Negative, Positive)")

print("\n" + "="*70)

```

```

=====
LOADING PRE-TRAINED MODEL
=====

📦 Loading model: distilbert-base-uncased-finetuned-sst-2-english
config.json: 100% 629/629 [00:00<00:00, 14.5kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 998B/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 3.95MB/s]
  ✓ Tokenizer loaded
model.safetensors: reconstructing file: 100% 268MB / 268MB, 21.6MB/s
model.safetensors: downloading bytes: 250MB, 17.8MB/s
Loading weights: 100% 104/104 [00:00<00:00, 1442.21it/s]
  ✓ Model loaded

📄 Model details:
  Number of parameters: 66,955,010
  Number of labels: 2 (Negative, Positive)

=====

```

```

print("=*70)
print("TOKENIZING DATASET")
print("=*70)

def tokenize_function(examples):
    """
    Converts text to tokens (numbers) that the model understands.
    """
    return tokenizer(
        examples["text"],
        padding="max_length", # Pad shorter texts to max length
        truncation=True,      # Cut longer texts at max length
        max_length=512        # Maximum token length
    )

print("\n📦 Tokenizing training set...")
tokenized_train = dataset['train'].map(tokenize_function, batched=True)

print("\n📦 Tokenizing test set...")
tokenized_test = dataset['test'].map(tokenize_function, batched=True)

print("\n✅ Tokenization complete!")

# Show example
print(f"\n🔍 Example tokenization:")
print(f"  Original text: {dataset['train'][0]['text']}")
print(f"  Label: {dataset['train'][0]['label']}")
print(f"  Tokenized (first 20 tokens): {tokenized_train[0]['input_ids'][:20]}")
print(f"  Total tokens: {len(tokenized_train[0]['input_ids'])}")

print("\n" + "=*70)

```

```

=====
TOKENIZING DATASET
=====

📦 Tokenizing training set...
Map: 100% 41/41 [00:00<00:00, 246.03 examples/s]

📦 Tokenizing test set...
Map: 100% 11/11 [00:00<00:00, 130.26 examples/s]

  ✓ Tokenization complete!

🔍 Example tokenization:
  Original text: Exceeded my expectations in every way
  Label: 1
  Tokenized (first 20 tokens): [101, 14872, 2026, 10908, 1999, 2296, 2126, 102, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
  Total tokens: 512

=====

```

```

print("=*70)
print("CONFIGURING TRAINING PARAMETERS")
print("=*70)

```

```

training_args = TrainingArguments(
    output_dir="./results",                # Where to save checkpoints
    num_train_epochs=3,                    # Train for 3 complete passes
    per_device_train_batch_size=8,         # Process 8 examples at once
    per_device_eval_batch_size=8,         # Evaluate 8 examples at once
    learning_rate=2e-5,                   # How aggressively to update weights
    weight_decay=0.01,                    # Regularization to prevent overfitting
    eval_strategy="epoch",                 # Evaluate after each epoch
    save_strategy="epoch",                 # Save model after each epoch
    load_best_model_at_end=True,           # Load best performing model at end
    logging_steps=10,                      # Log every 10 steps
    seed=42,                              # For reproducibility
)

print("\n✅ Training configuration:")
print(f"  Epochs: {training_args.num_train_epochs}")
print(f"  Batch size: {training_args.per_device_train_batch_size}")
print(f"  Learning rate: {training_args.learning_rate}")
print(f"  Evaluation: After each epoch")

print("\n📊 Training will involve:")
train_steps = len(tokenized_train) // training_args.per_device_train_batch_size * training_args.num_train_epochs
print(f"  Approximately {train_steps} training steps")
print(f"  {len(tokenized_train) // training_args.per_device_train_batch_size} steps per epoch")

print("\n" + "="*70)

```

```

=====
CONFIGURING TRAINING PARAMETERS
=====

✅ Training configuration:
  Epochs: 3
  Batch size: 8
  Learning rate: 2e-05
  Evaluation: After each epoch

📊 Training will involve:
  Approximately 15 training steps
  5 steps per epoch

=====

```

```

print("="*70)
print("STARTING FINE-TUNING")
print("="*70)

# Define evaluation metric
def compute_metrics(eval_pred):
    """
    Calculates accuracy during training.
    """
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return {'accuracy': accuracy_score(labels, predictions)}

# Create Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    compute_metrics=compute_metrics,
)

print("\n🔥 Beginning fine-tuning...")
print("  This will take 2-5 minutes depending on GPU allocation.")
print("  You'll see progress updates below:\n")

# Start training!
trainer.train()

print("\n" + "="*70)
print("✅ FINE-TUNING COMPLETE!")
print("="*70)

```

```
=====
STARTING FINE-TUNING
=====
```

🔥 Beginning fine-tuning...  
This will take 2-5 minutes depending on GPU allocation.  
You'll see progress updates below:

/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin\_memory' argument is set as true but no pin\_memory\_device is available. This warning can be disabled by setting 'pin\_memory\_device=-1' in the DataLoader constructor.  
super().\_\_init\_\_(loader)

[18/18 08:13, Epoch 3/3]

| Epoch | Training Loss | Validation Loss | Accuracy |
|-------|---------------|-----------------|----------|
| 1     | No log        | 0.434365        | 0.909091 |
| 2     | 1.121446      | 0.383929        | 0.909091 |
| 3     | 1.121446      | 0.371158        | 0.909091 |

Writing model shards: 100% 1/1 [00:05<00:00, 5.76s/it]

/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin\_memory' argument is set as true but no pin\_memory\_device is available. This warning can be disabled by setting 'pin\_memory\_device=-1' in the DataLoader constructor.  
super().\_\_init\_\_(loader)

Writing model shards: 100% 1/1 [00:01<00:00, 1.73s/it]

/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin\_memory' argument is set as true but no pin\_memory\_device is available. This warning can be disabled by setting 'pin\_memory\_device=-1' in the DataLoader constructor.  
super().\_\_init\_\_(loader)

Writing model shards: 100% 1/1 [00:06<00:00, 6.01s/it]

=====

✅ FINE-TUNING COMPLETE!

=====

```
print("="*70)
print("SAVING FINE-TUNED MODEL")
print("="*70)

# Save model and tokenizer
output_dir = "./fine_tuned_sentiment_model"

print(f"\n💾 Saving model to: {output_dir}")

model.save_pretrained(output_dir)
tokenizer.save_pretrained(output_dir)

print("✅ Model saved!")
print("✅ Tokenizer saved!")

print(f"\n🔍 Saved files:")
import os
for file in os.listdir(output_dir):
    file_size = os.path.getsize(os.path.join(output_dir, file)) / (1024*1024) # Size in MB
    print(f"  - {file} ({{file_size:.2f}} MB)")

# Create a zip file
print(f"\n📦 Creating zip file...")
!zip -r fine_tuned_sentiment_model.zip {output_dir}

print("✅ Zip file created!")

# Download the zip file
print(f"\n📄 Downloading model...")
from google.colab import files
files.download('fine_tuned_sentiment_model.zip')

print("✅ Download started!")
print("\n💡 The zip file should download to your computer's Downloads folder.")
print("  You can extract it and use it later by loading it with:")
print("  model = AutoModelForSequenceClassification.from_pretrained('path/to/extracted/folder')")
print("  tokenizer = AutoTokenizer.from_pretrained('path/to/extracted/folder')")

print("\n" + "="*70)
```

```
=====
SAVING FINE-TUNED MODEL
=====

Saving model to: ./fine_tuned_sentiment_model
Writing model shards: 100% 1/1 [00:05<00:00, 5.23s/it]
  ✓ Model saved!
  ✓ Tokenizer saved!

Saved files:
- config.json (0.00 MB)
- tokenizer.json (0.68 MB)
- tokenizer_config.json (0.00 MB)
- model.safetensors (255.43 MB)

Creating zip file...
adding: fine_tuned_sentiment_model/ (stored 0%)
adding: fine_tuned_sentiment_model/config.json (deflated 49%)
adding: fine_tuned_sentiment_model/tokenizer.json (deflated 71%)
adding: fine_tuned_sentiment_model/tokenizer_config.json (deflated 43%)
adding: fine_tuned_sentiment_model/model.safetensors (deflated 8%)
  ✓ Zip file created!
```